Design Tradeoffs for SSD Performance

Agrawal, Nitin, Ted Wobber, John D. Davis, Mark Manasse, Rina Panigrahy

Microsoft Research, Silicon Valley University of Wisconsin-Madison

2008 USENIX Annual Technical Conference

2026. 07. 28.

Presentation by Sung JinWook, Jung HuiChan

sungjw0408@dankook.ac.kr, a011214@dankook.ac.kr

Contents

1. Introduction

- **The Hidden Architecture**: SSD internals as trade secrets
- **Goal**: Analyze core design trade-offs
- **Impact**: Performance variations across workloads

2. Background

- **Shared Resources**
  - 8-bit I/O Bus,
  - Control signals
- **Independent Signals**
  - Chip Enable (CE)
  - Ready/Busy signals per die

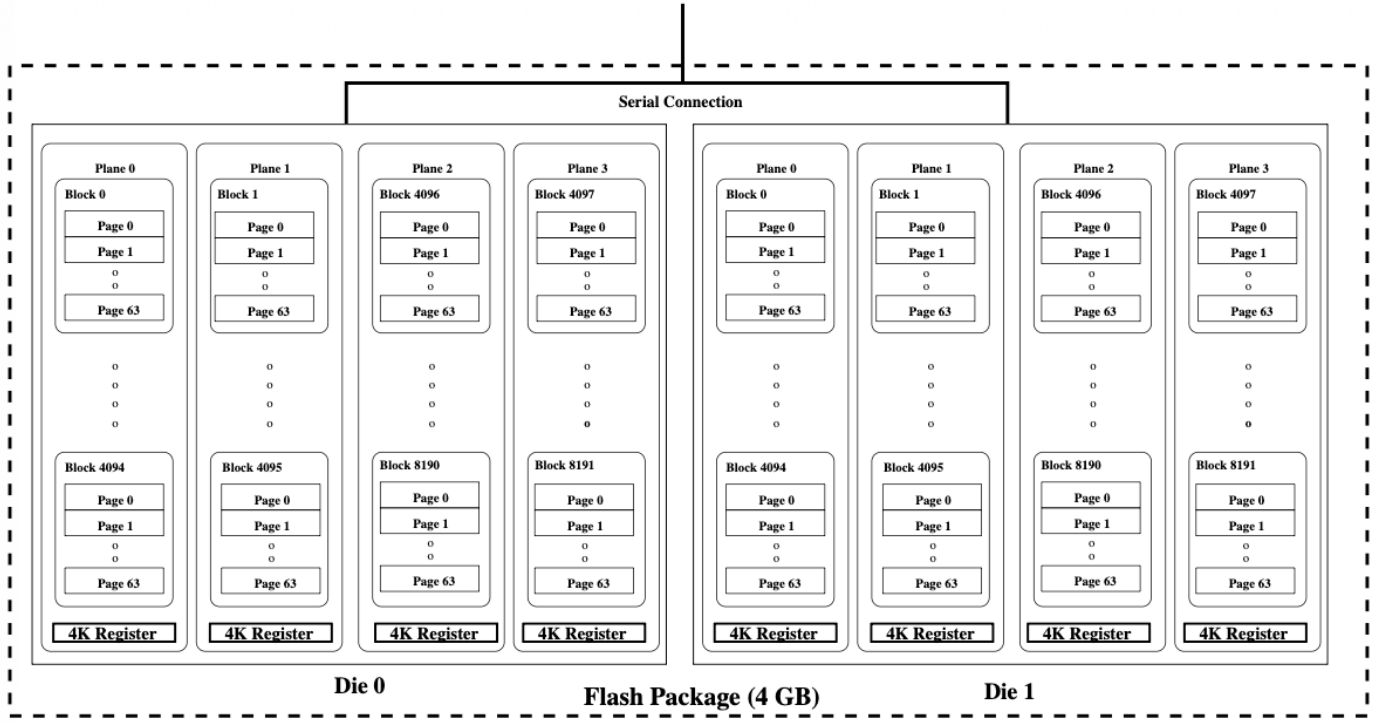


Figure 1: Samsung 4GB flash internals

2. Background

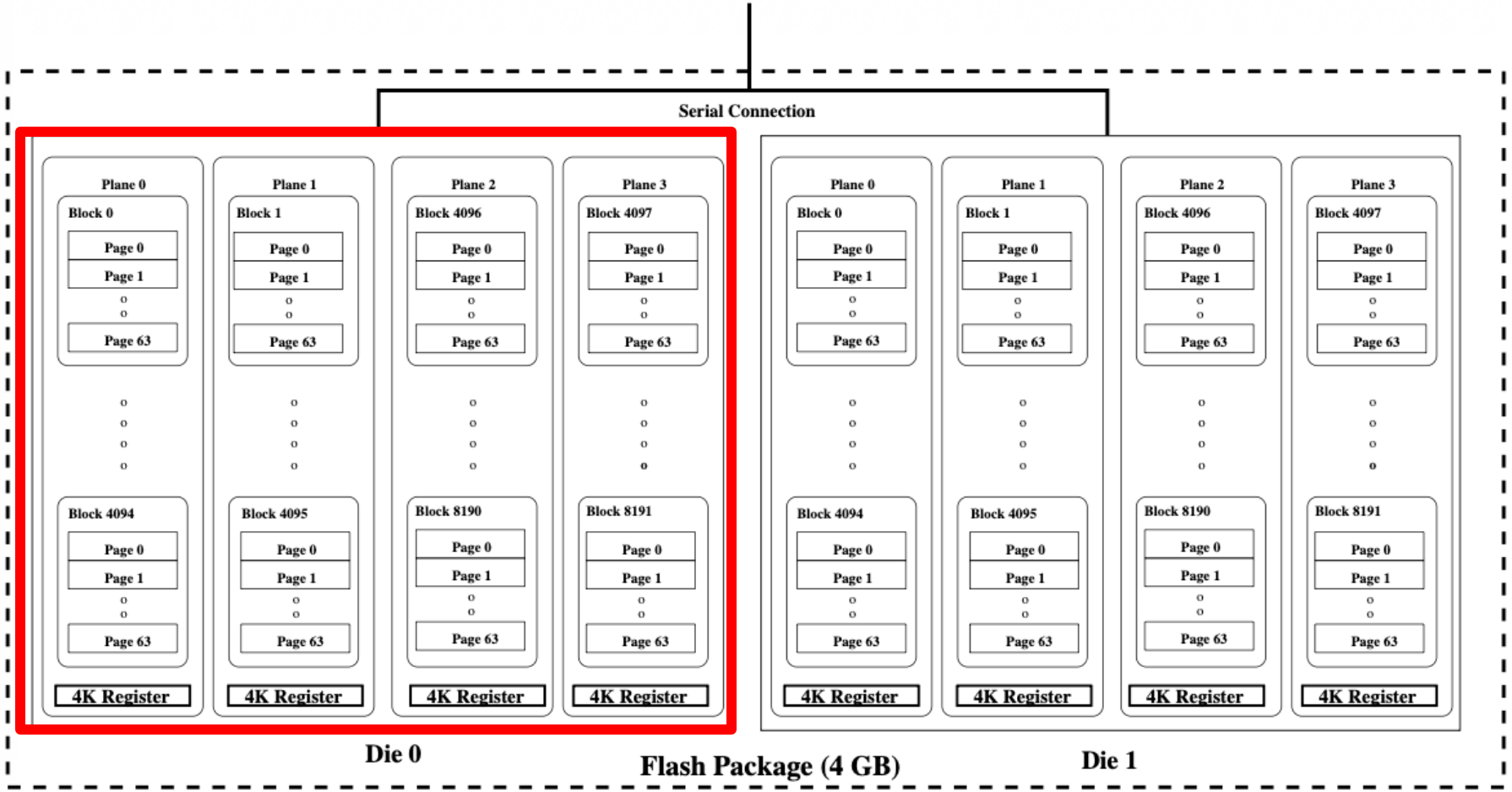


Figure 1: Samsung 4GB flash internals

Each die contains 8,192 blocks

2. Background

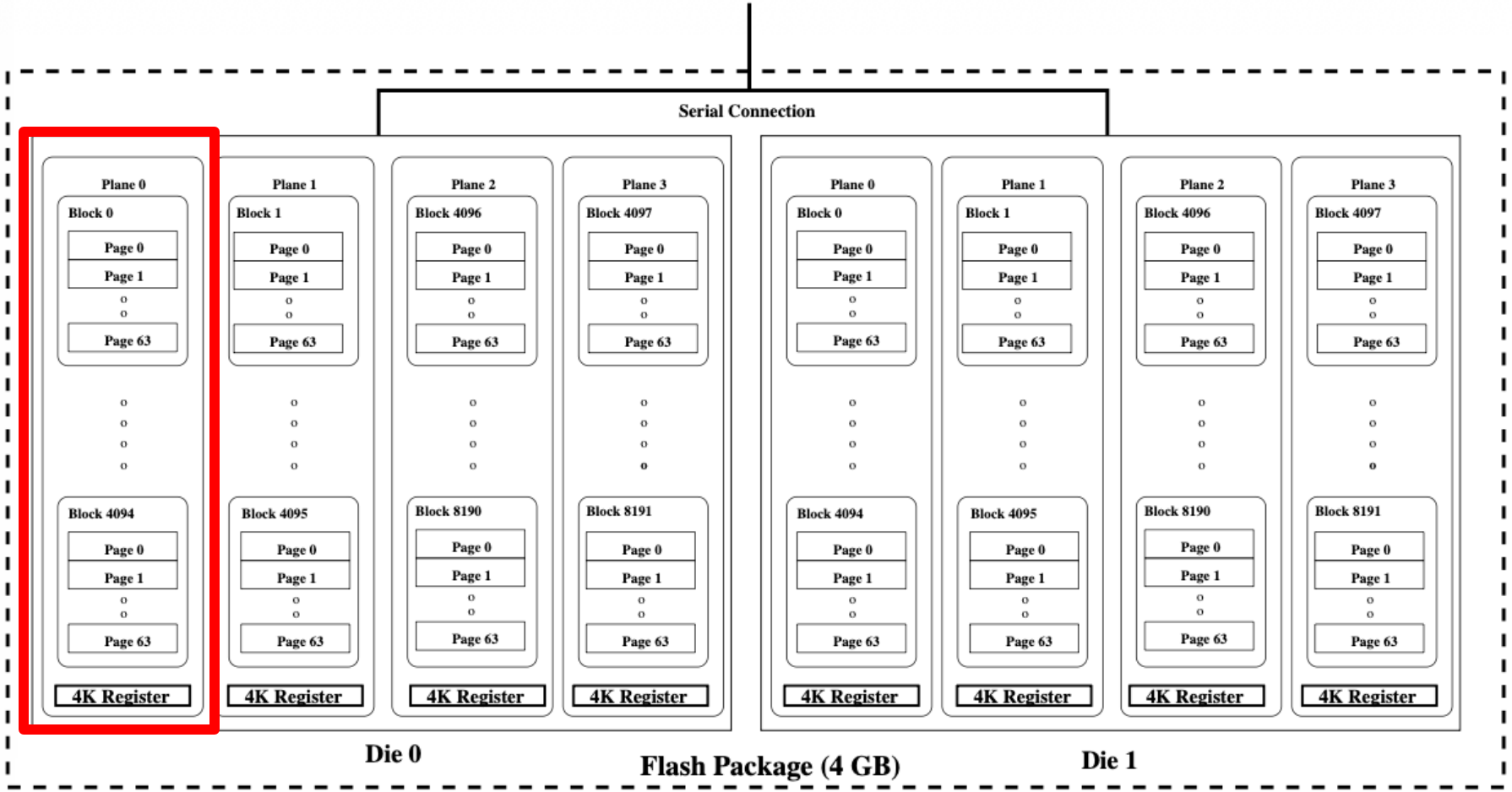


Figure 1: Samsung 4GB flash internals

blocks are divided into 4 planes.

2. Background

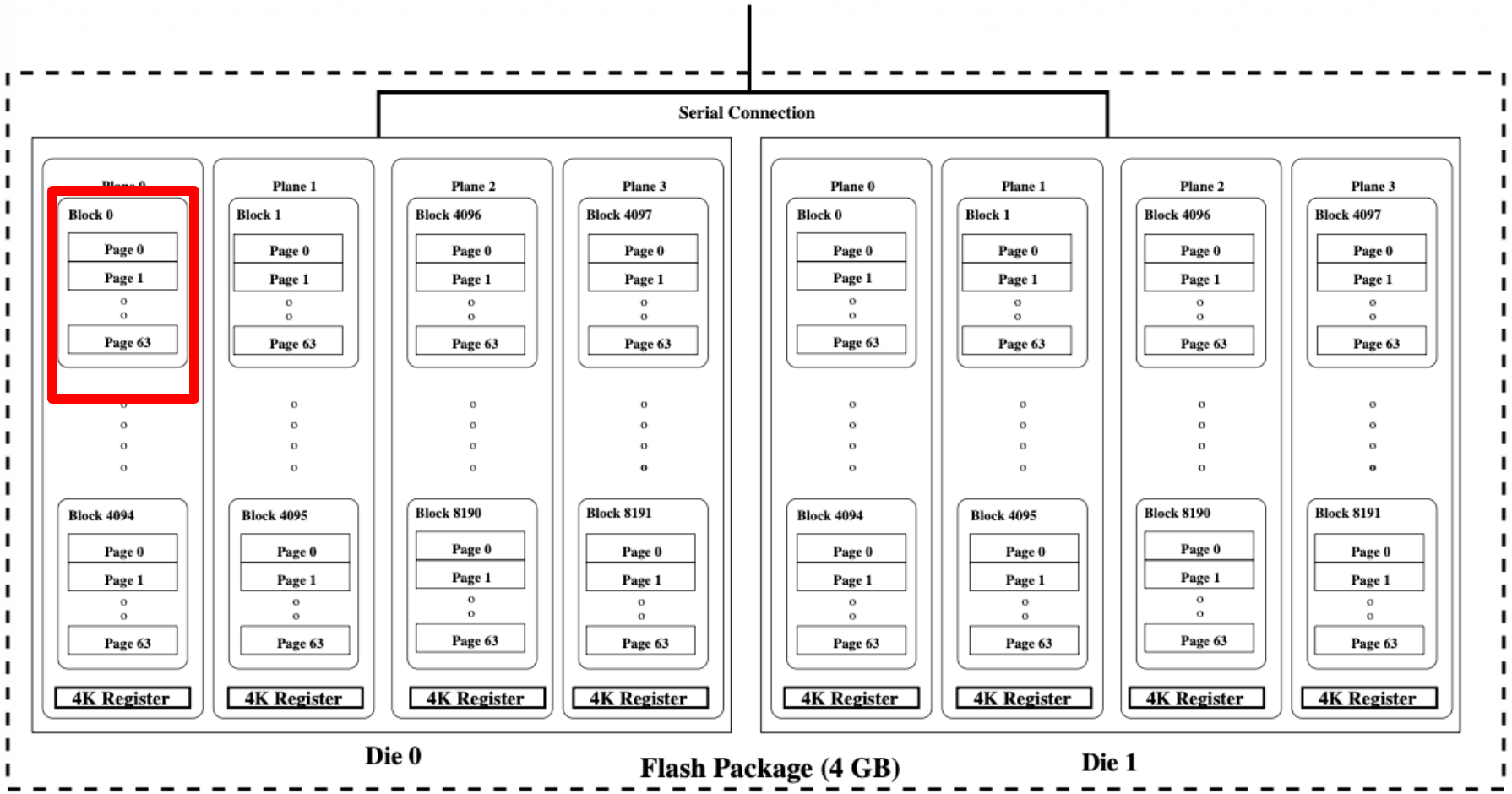


Figure 1: Samsung 4GB flash internals

Each block consists of 64 pages

2.1. Properties of Flash Memory

- **Read**
  - Page-level access
  - 25μs + 100μs
- **Erase**
  - Block-level access
  - 1.5ms(1500μs)
  - No in-place update: Must erase before writing new data
  - Limitation: Finite erase cycles require Wear-leveling

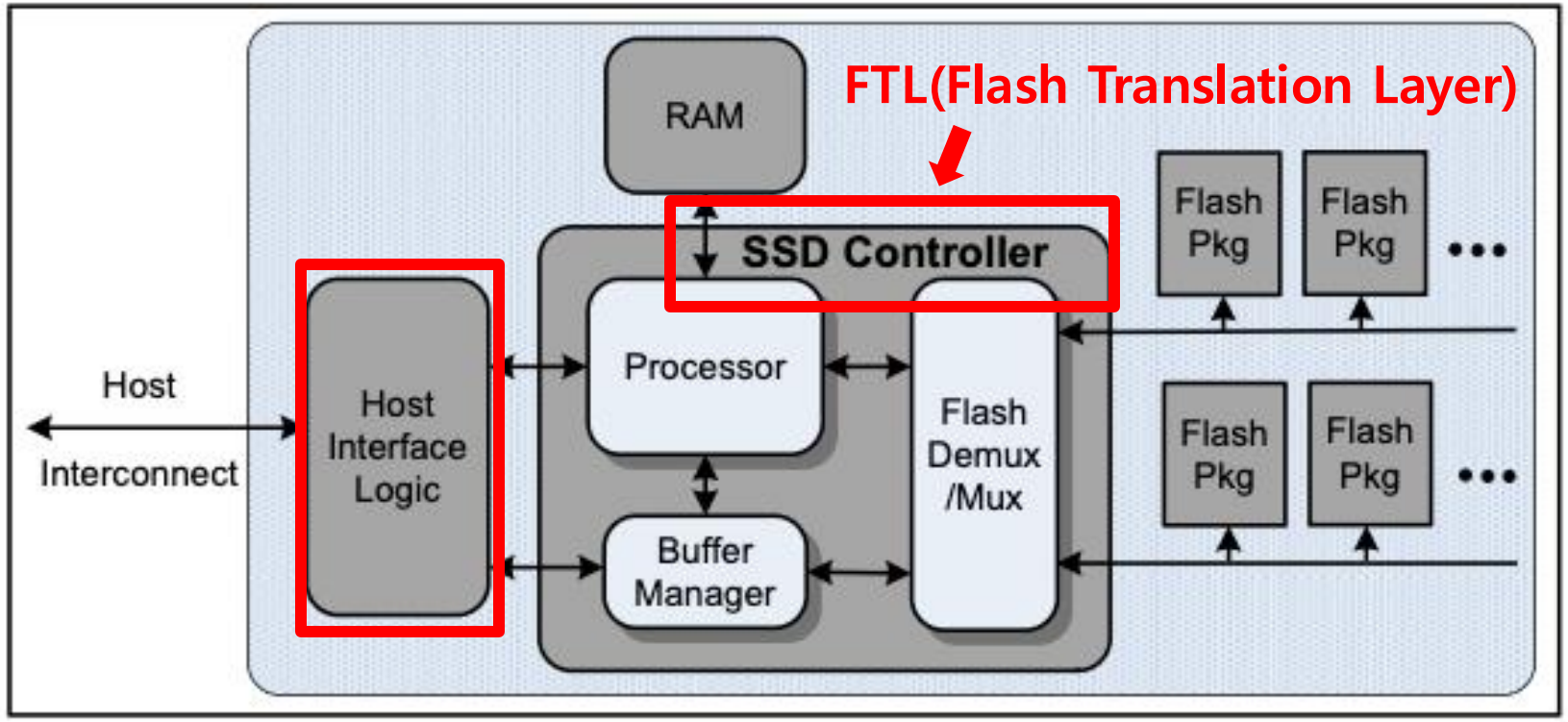
2.1. Properties of Flash Memory

- **Write(program)**
  - Page-level access
  - 100μs + 200μs
  - Sequential Constraint
  - Copy-back: Fast internal data copy bypassing the external buffer
  - Each 4KB page includes 128B of extra space for metadata

2.2. Bandwidth and Interleaving

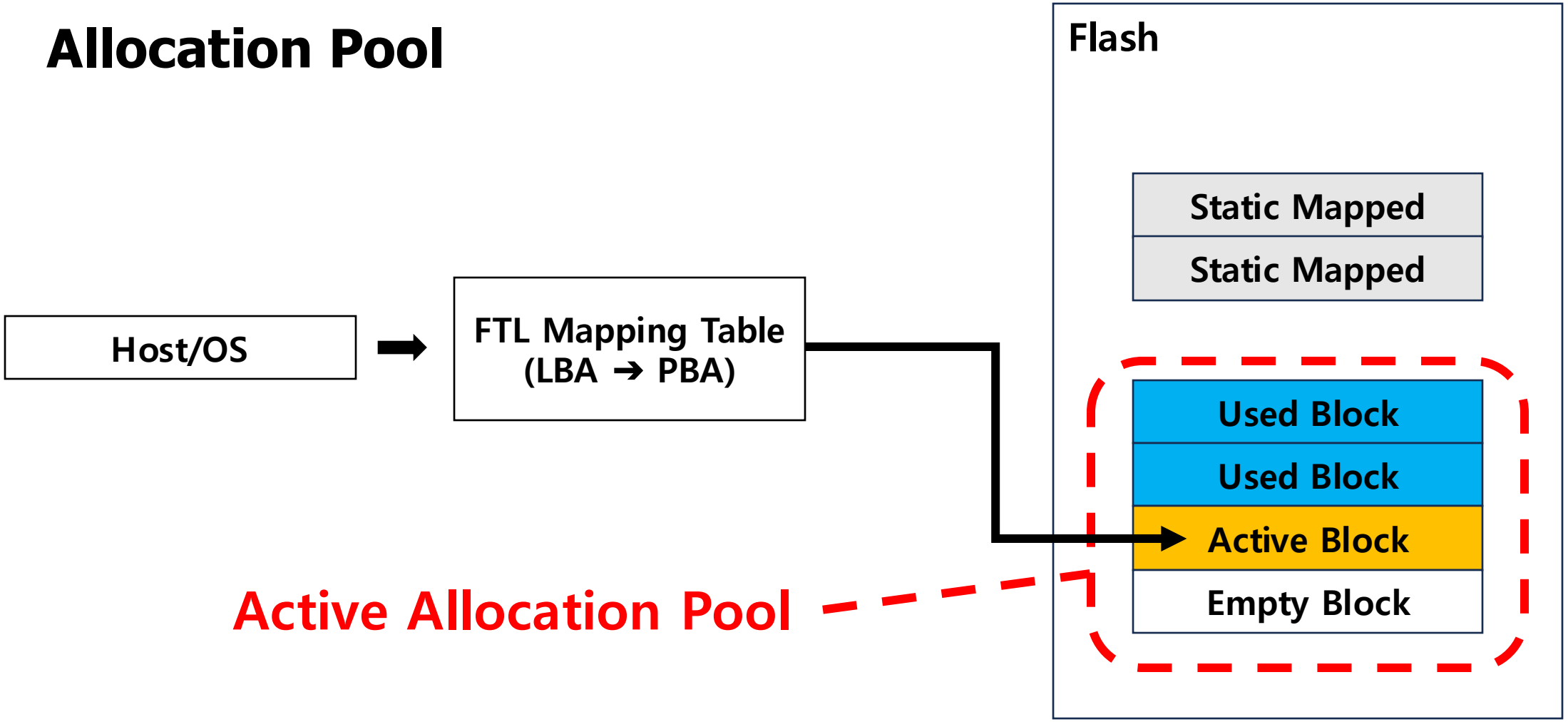
- **Bottleneck: Serial interface**
- **Interleaving**
  - Constraints
    - Operations within the same flash Plane cannot be interleaved.
    - Copy-back: Fast as it bypasses external serial pins, but must be executed strictly within the same Plane.

3. SSD Basics


FTL(Flash Translation Layer)
RAM
SSD Controller
Host
Interconnect
Host Interface Logic
Processor
Buffer Manager
Flash Demux /Mux
Flash Pkg
Flash Pkg
Flash Pkg
Flash Pkg

3.1. Logical Block Map

- **Allocation Pool**



3.1. Logical Block Map

- **Design Constraints**
  - Load Balancing
    - I/O operations evenly balanced
  - Parallel access
    - Mapping to a parallel accessible structure
  - Block erasure
    - Write after Erase

3.1. Logical Block Map

- **Dilemmas between design variables and constraints**

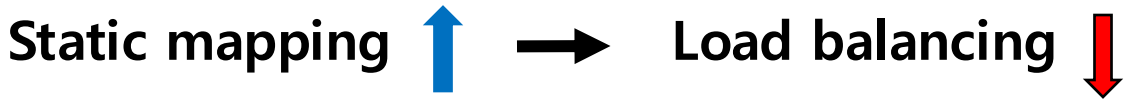
1. Static Mapping vs Load Balancing

2. Logical Page size

3.1. Logical Block Map

- **Dilemmas between design variables and constraints**

1. Static Mapping vs Load Balancing



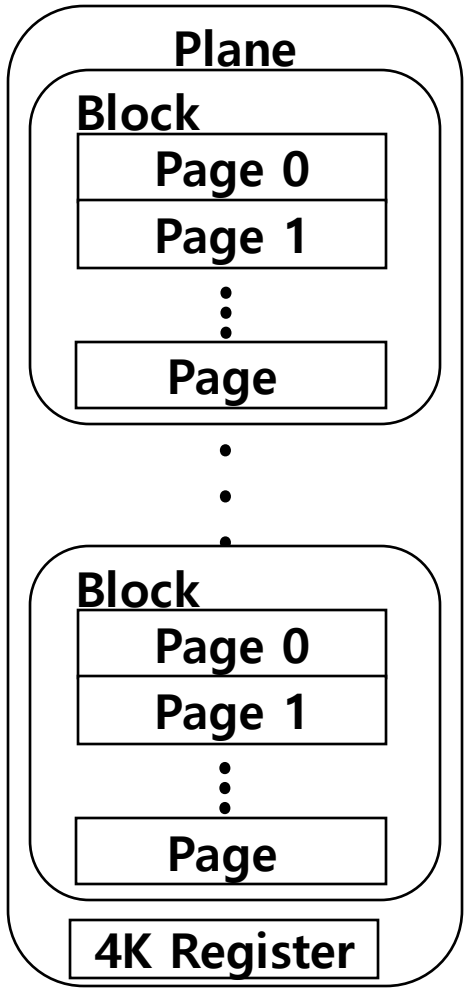
2. Logical Page size

3.1. Logical Block Map

- **Dilemmas between design variables and constraints**

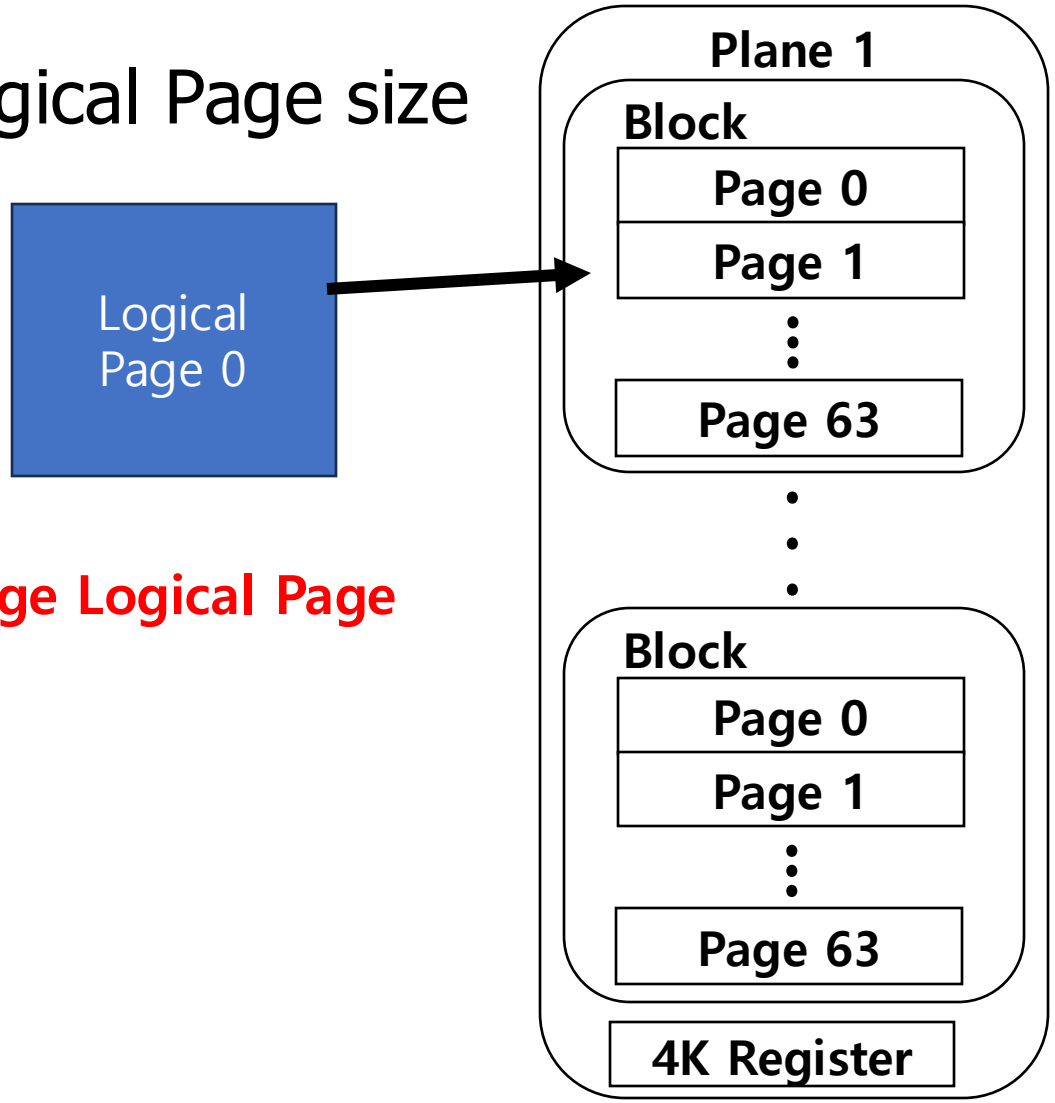
1. Static Mapping vs Load Balancing

2. Logical Page size

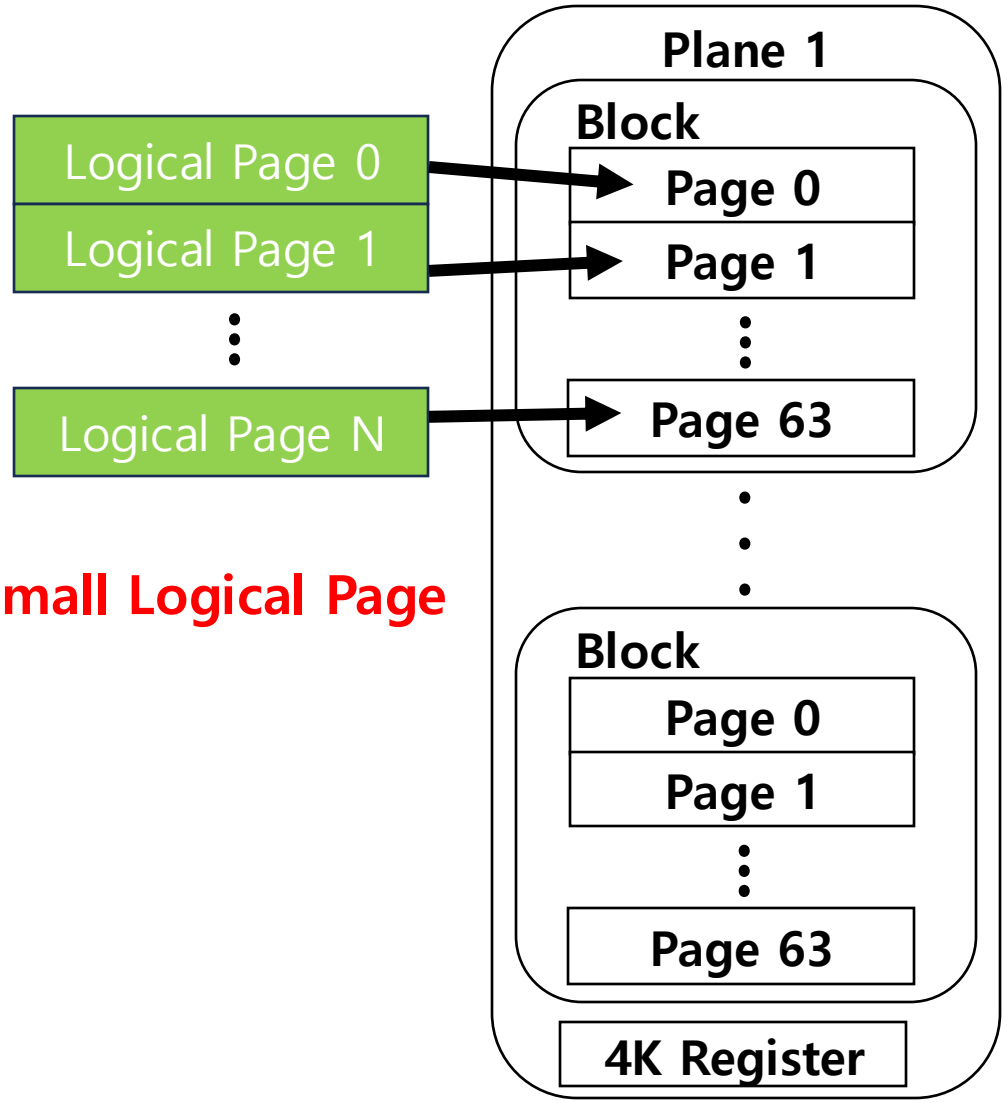


3.1. Logical Block Map

- Logical Page size



Large Logical Page



Small Logical Page

3.1. Logical Block Map

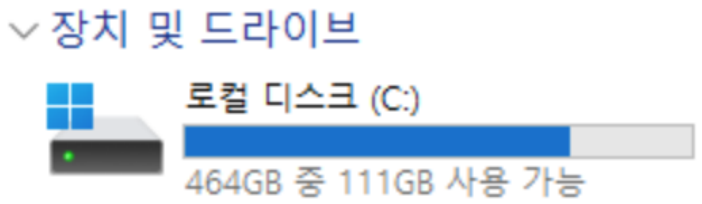
- **Fine Granularity Striping**
  - Optimal Size: 4KB
  - RAID 0 Principle
- **Result**
  - Parallel
  - Load balancing

3.2. Cleaning

- **Garbage Generation**
  - Overwriting creates outdated data
  - Superseded' pages
- **Data Relocation**
  - Performance Drop
- **Overprovisioning**

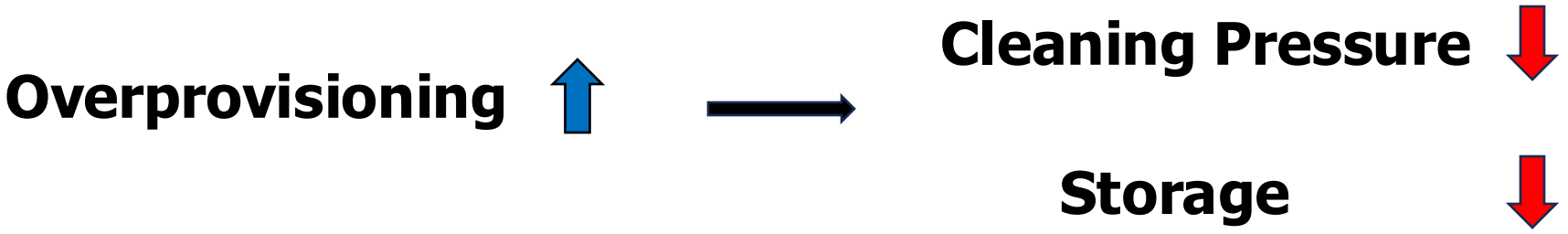
3.2. Cleaning

- **Overprovisioning**
  - A hidden reserve of spare capacity within the SSD
    - EX) 512GB SSD

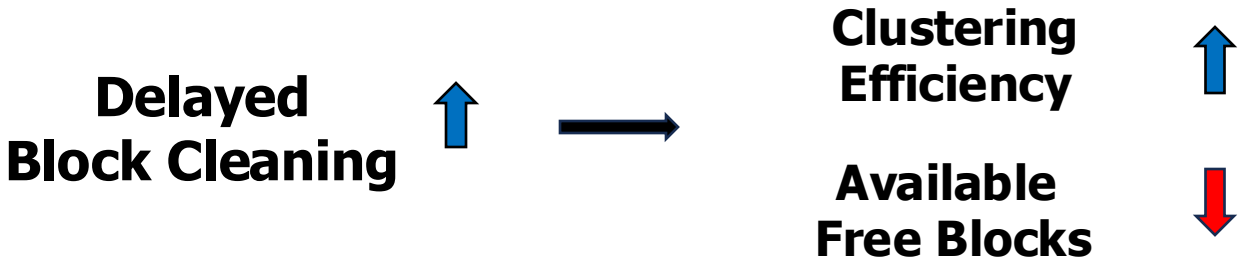


3.2. Cleaning

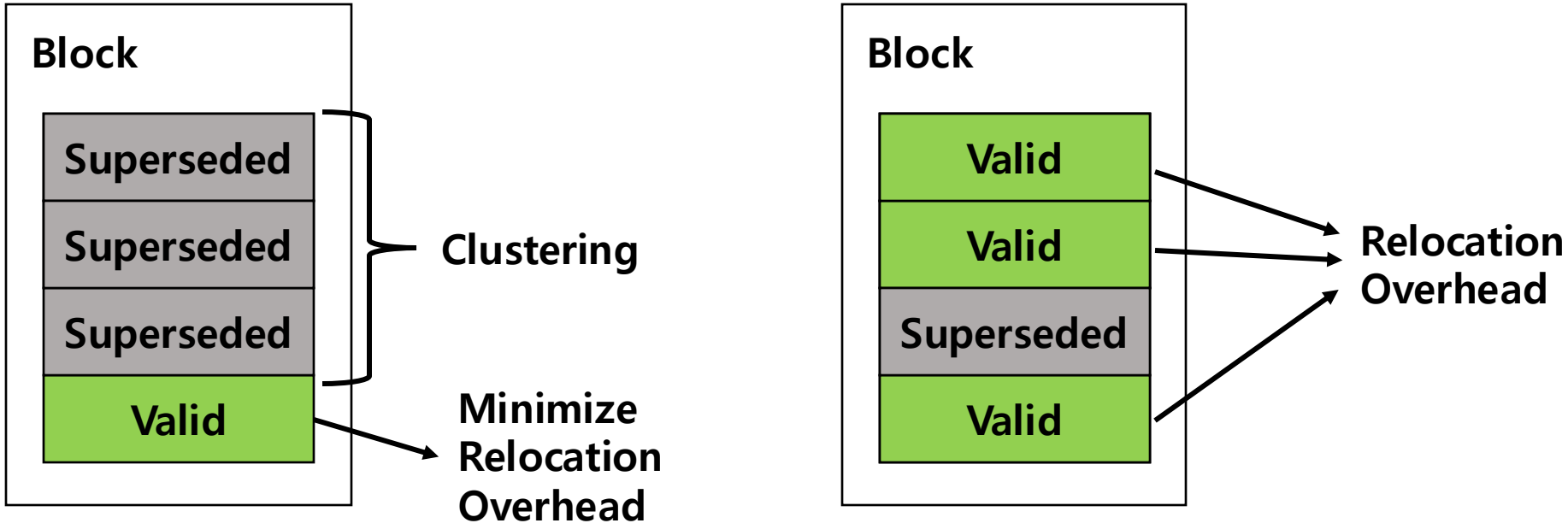
- **Trade-off 1**
- Overprovisioning: User Capacity vs. Garbage Collection Pressure



3.2. Cleaning

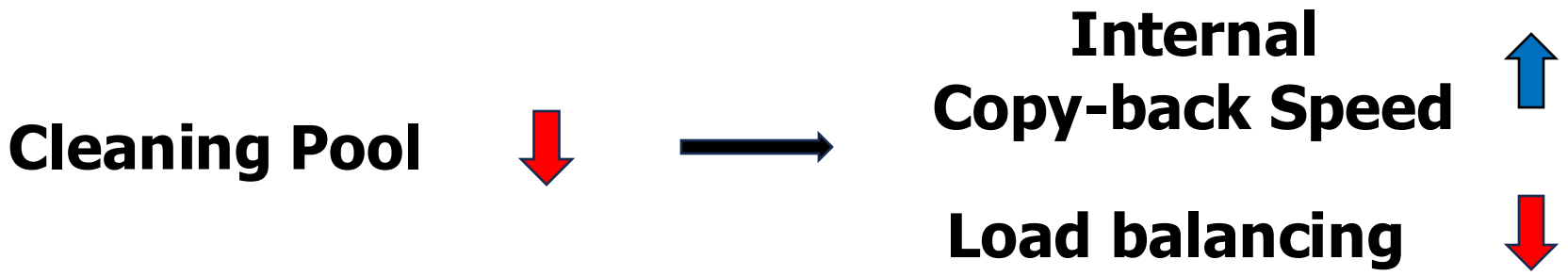


- **Trade-off 2**
- Delayed block cleaning: Free Space Reclamation vs. Clustering Efficiency



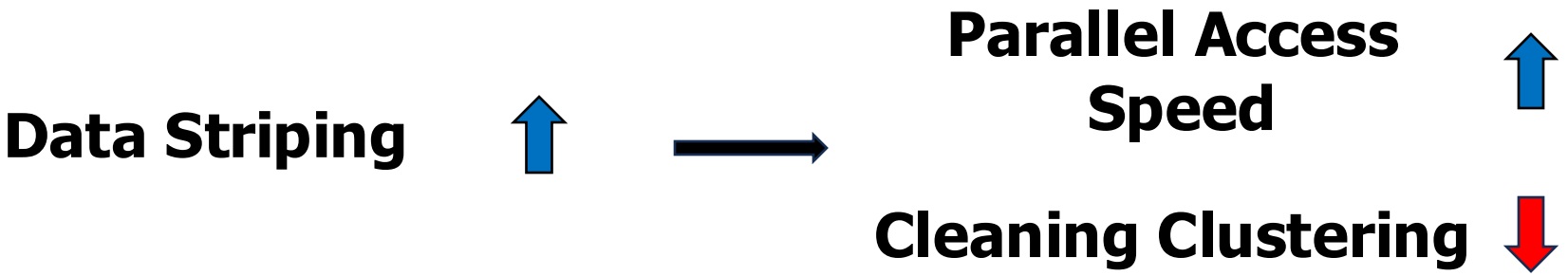
3.2. Cleaning

- **Trade-off 3**
- Allocation pool size: Load Balancing vs Copy-Back Performance



3.2. Cleaning

- **Trade-off 4**
- Data Striping: Parallel Access vs Cleaning Clustering



3.3. Parallelism and Interconnect Density

- **Parallelism**
  - handle I/O requests on multiple flash packages

- **Physical Pin Limits**
  - 32GB SSD (8 flash components) ➔ Requires 136 pins on the controller

- **Trade-off**
  - Full Connectivity
  - Shared-bus Gang
  - Shared-control Gang
  - Interleaving in Shared-bus

3.3. Parallelism and Interconnect Density

- **Full Connectivity**
  - Independent Control & Data Bus ⬆
    - Maximum Parallel Performance & Flexibility ⬆
    - Hardware Cost & Complexity (Pin Count) ⬆

unrealistic

3.3. Parallelism and Interconnect Density

- **Shared-bus Gang**
  - Shared Data Bus (Fewer Pins)
- **Shared-control Gang**

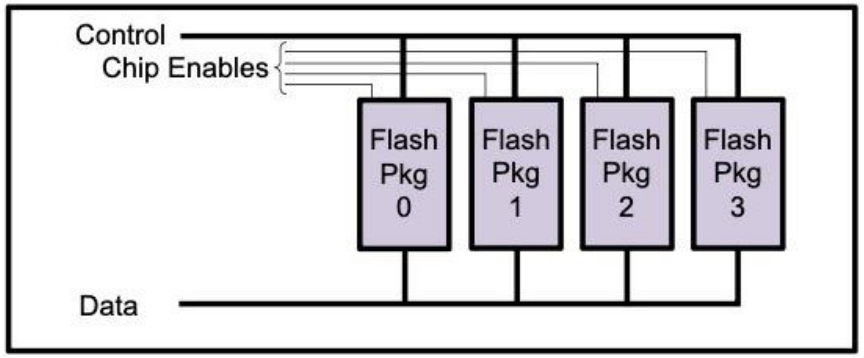


Figure 4: Shared bus gang

3.3. Parallelism and Interconnect Density

- **Shared-bus Gang**
  - Shared Data Bus (Fewer Pins)

- **Shared-control Gang**
  - Shared Control Pins (Synchronized Execution)

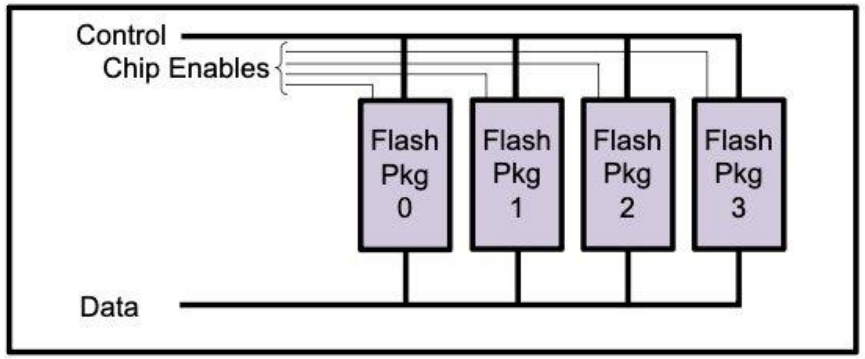


Figure 4: Shared bus gang

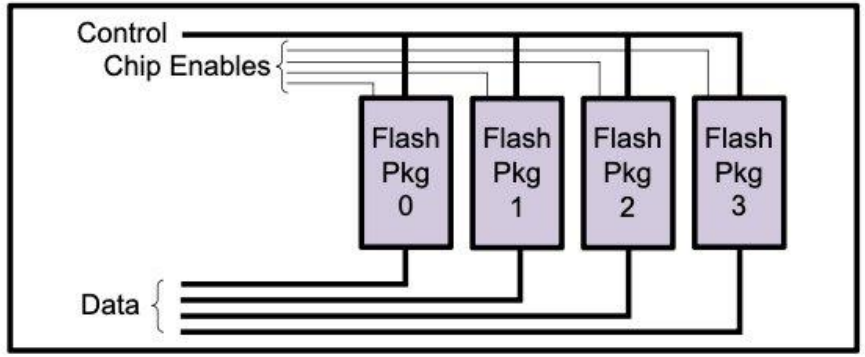


Figure 5: Shared control gang

3.3. Parallelism and Interconnect Density

- **Interleaving in Shared-bus**
  - Interleaving Operations (Software optimization) ↑
  - Hides latency of slow operations (e.g., Erase) ↑
  - Eventually bounded by physical shared-bus limits (Bus contention) ↓

3.3. Parallelism and Interconnect Density

Conclusion

- No single hardware configuration is optimal for all workloads.
  - Highly Sequential Workloads ➔ Shared-control Ganging
  - Workloads with Inherent Parallelism ➔ Full Connectivity
  - Workloads with Poor Locality ➔ Sophisticated Cleaning Strategies(e.g. copy-back)

4. Evaluation: No Single Solution

4. Evaluation

- Environment
  - **Workload**
    - TPC-C: Random Requests (Poor Locality)
    - Exchange: Random Requests (Some Locality)
    - IOzone & Postmark: Sequential Requests
  - Simulation
    - Modified Version of DiskSim Simulator
      (for HDD → SSD)

4. Evaluation

- SSD Performance Under Microbenchmarks
  - When GC?

Microbenchmark	Cleaning	Latency (μs)	IO/s
Sequential read	x	130	61,255
Random read	x	130	61,255
Sequential write	x	309	25,898
Random write	x	309	25,898
Sequential write	√	327	24,457
Random write	√	433	18,480

Table 3: SSD performance under microbenchmarks

4. Evaluation

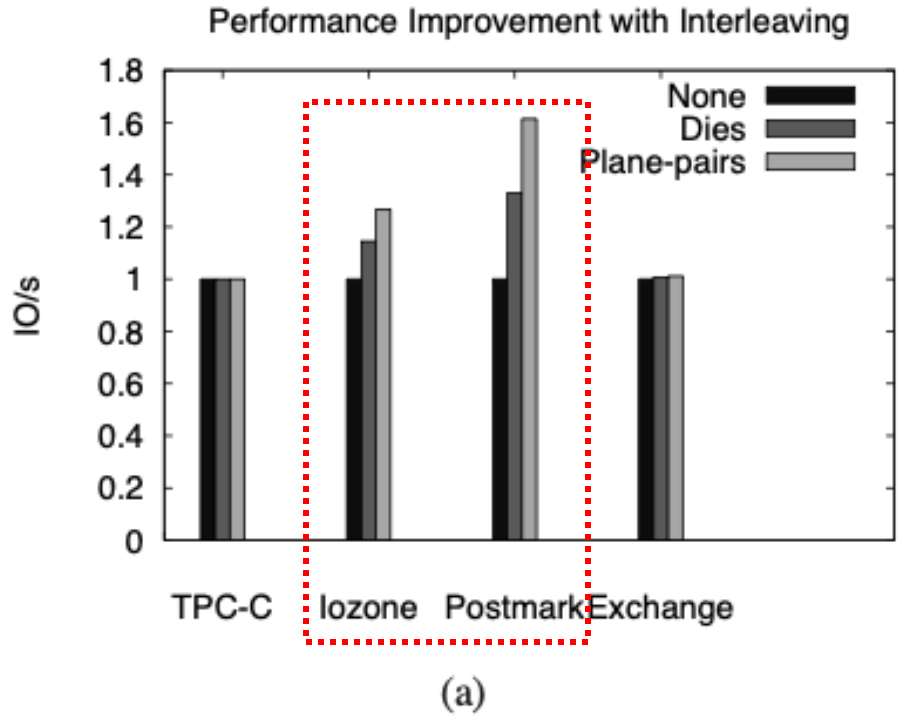
- Page Size on I/O Latency
  - Workload: TPC-C
    - Page Size 256KB
      - I/O Latency: 20ms → Write Amplification ↑
    - Page Size 4KB
      - I/O Latency: 200μs(=0.2ms)

4. Evaluation

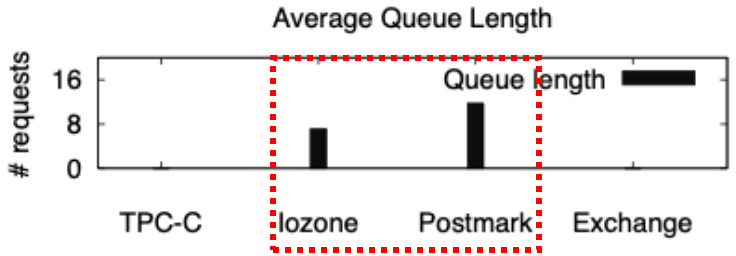
- Parallelism Mechanism (Interleaving & Ganging)
  - In Which Workload is Interleaving Effective?

4. Evaluation

- Parallelism Mechanism (Interleaving & Ganging)
  - In Which Workload is Interleaving Effective?



(a)



(b)

Figure 6: Impact of interleaving

4. Evaluation

- Parallelism Mechanism (Interleaving & Ganging)
  - Asynchronous vs. Synchronous Ganging
    - IOzone/Postmark's Response Time ↑

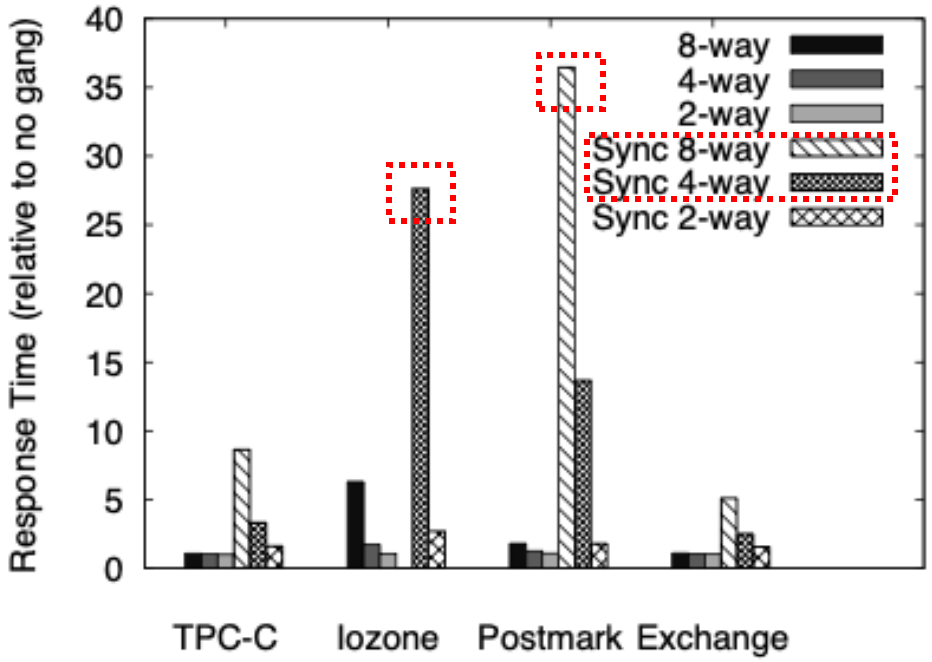


Figure 7: Shared-control ganging

4. Evaluation

- Block Cleaning Mechanism
  - Inter-plane Transfer vs. Copy-back
    1) Workload: TPC-C
    2) Workload: IOzone/Postmark

4. Evaluation

- Block Cleaning Mechanism
  - Inter-plane Transfer vs. Copy-back
    1) Workload: TPC-C
    2) Workload: IOzone/Postmark

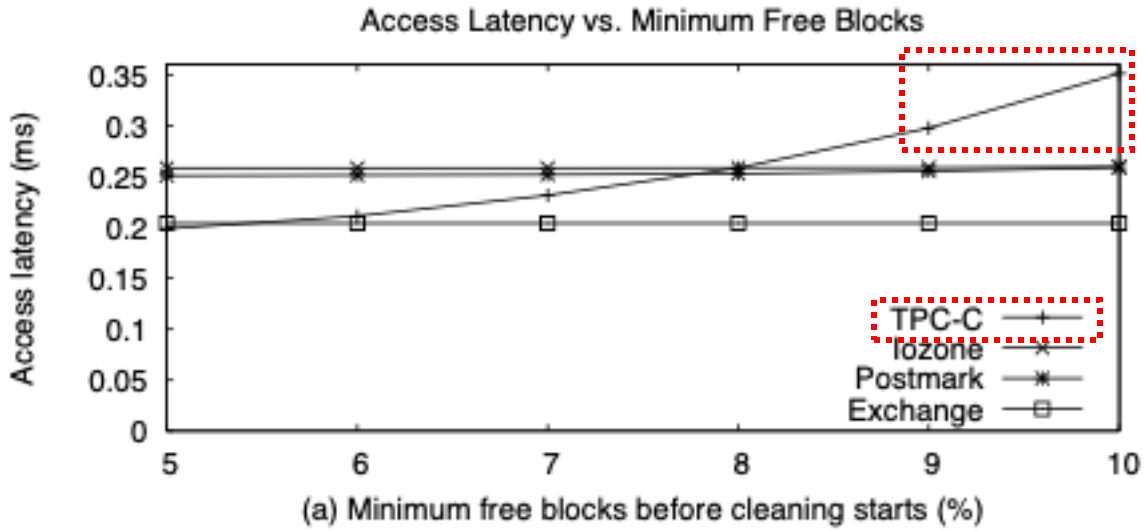
	# cleaned	Avg. time (ms)	Efficiency
TPC-C (inter-plane)	114	9.65	70%
TPC-C (copy-back)	108	5.85	70%
IOzone	101170	1.5	100%
Postmark	2693	1.5	100%

40% ↓

Table 5: Cleaning frequency and efficiency

4. Evaluation

- Cleaning (GC) Thresholds Optimization
  - Why did TPC-C's Access Latency Explode? Poor Locality



4. Evaluation

- Summary: No Single Solution for SSD Performance

4. Evaluation

- Summary: No Single Solution for SSD Performance
  - Hardware
  - Workload
  - Software (FTL Mechanism)

4. Evaluation

- Summary: No Single Solution for SSD Performance
  - Hardware
  - Workload
  - Software (FTL Mechanism)

Technique	Positives	Negatives
Large allocation pool	Load balancing	Few intra-chip ops
Large page size	Small page table	Read-modify-writes
Overprovisioning	Less cleaning	Reduced capacity
Ganging	Sparser wiring	Reduced parallelism
Striping	Concurrency	Loss of locality

Table 6: SSD Design Tradeoffs in Brief

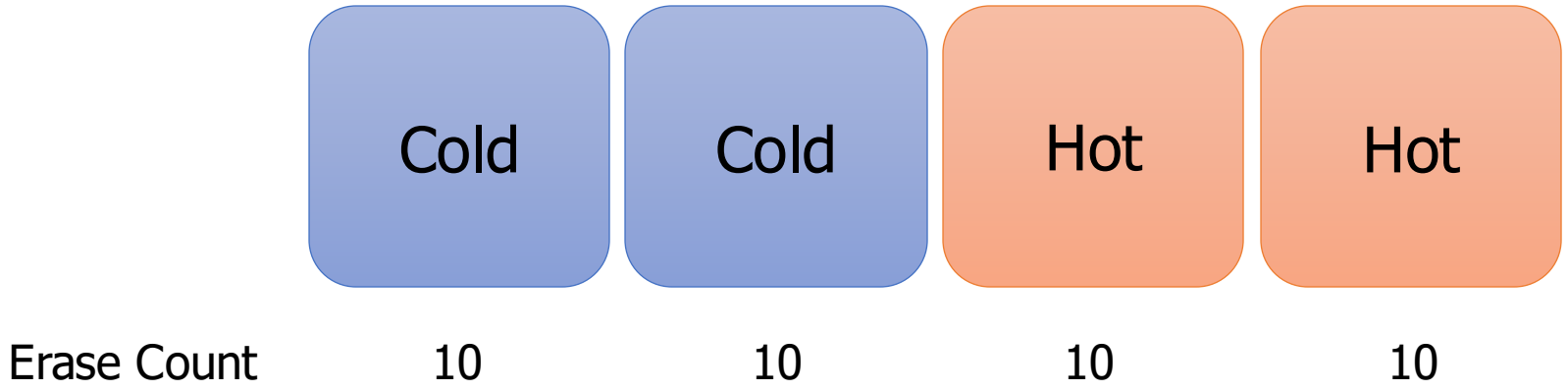
5. Overcoming Trade-offs: OS-FTL Co-design

5.1. The Dilemma of FTL Wear-leveling

- Greedy Approach
  - FTL Cleaning Efficiency ↑
  - Uneven Wear of SSD Blocks

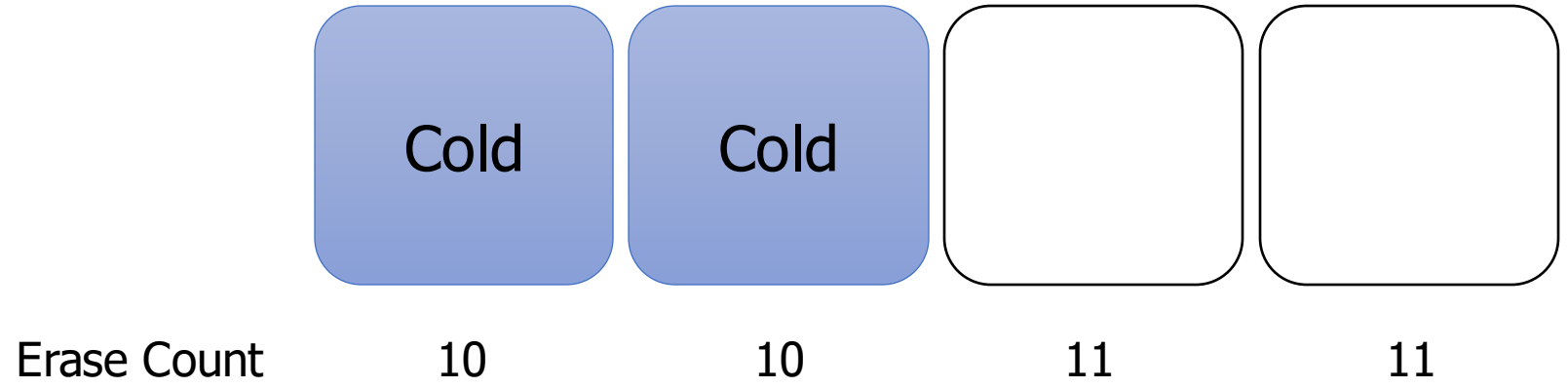
5.1. The Dilemma of FTL Wear-leveling

- Greedy Approach
  - FTL Cleaning Efficiency ↑
  - Uneven Wear of SSD Blocks



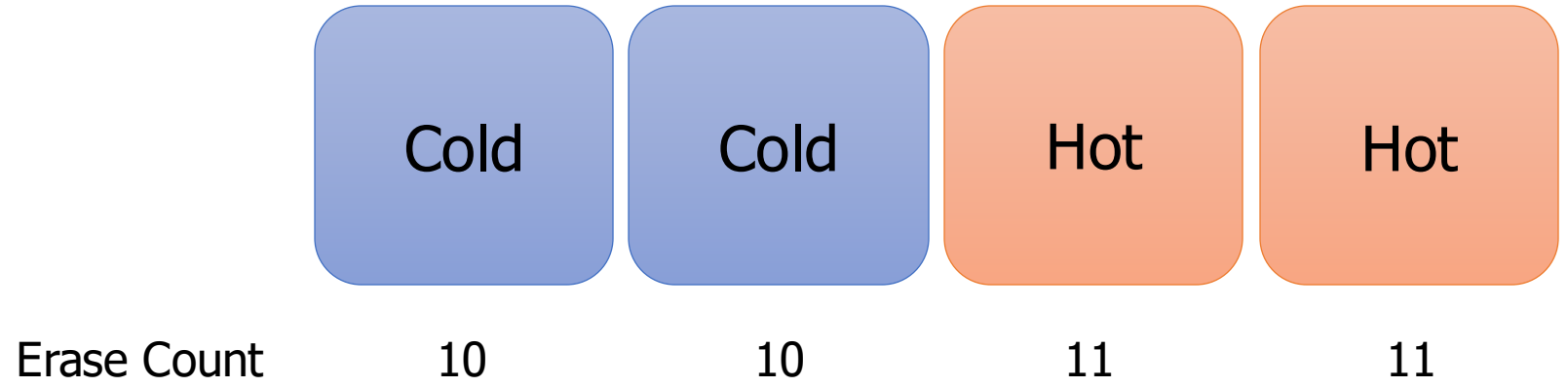
5.1. The Dilemma of FTL Wear-leveling

- Greedy Approach
  - FTL Cleaning Efficiency ↑
  - Uneven Wear of SSD Blocks



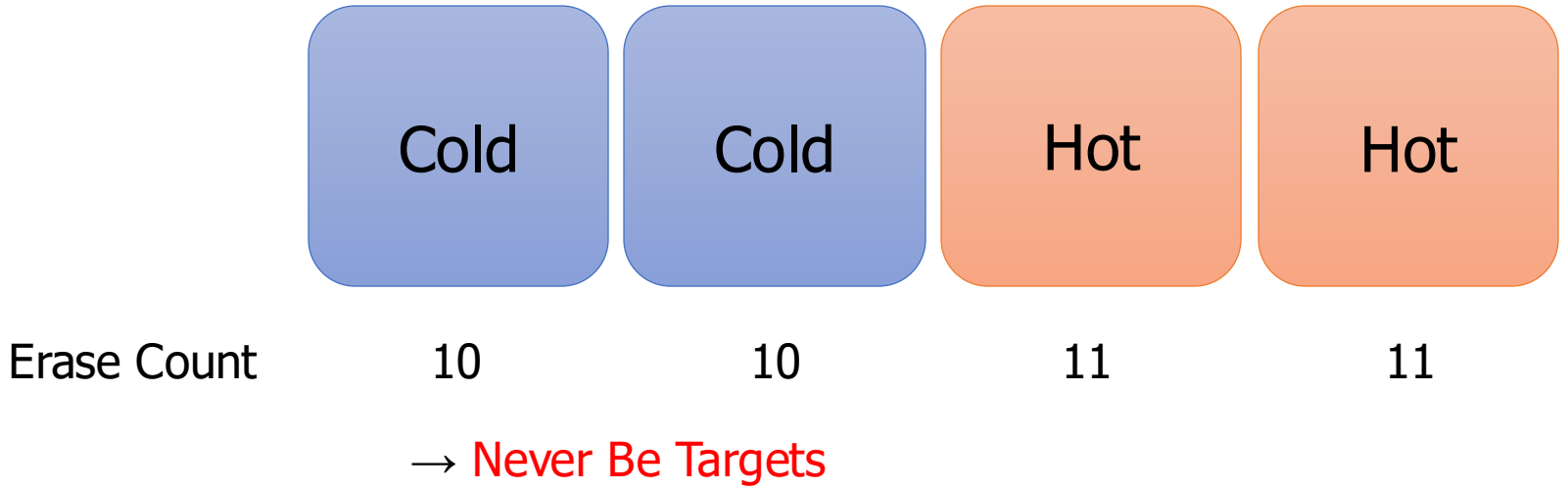
5.1. The Dilemma of FTL Wear-leveling

- Greedy Approach
  - FTL Cleaning Efficiency ↑
  - Uneven Wear of SSD Blocks



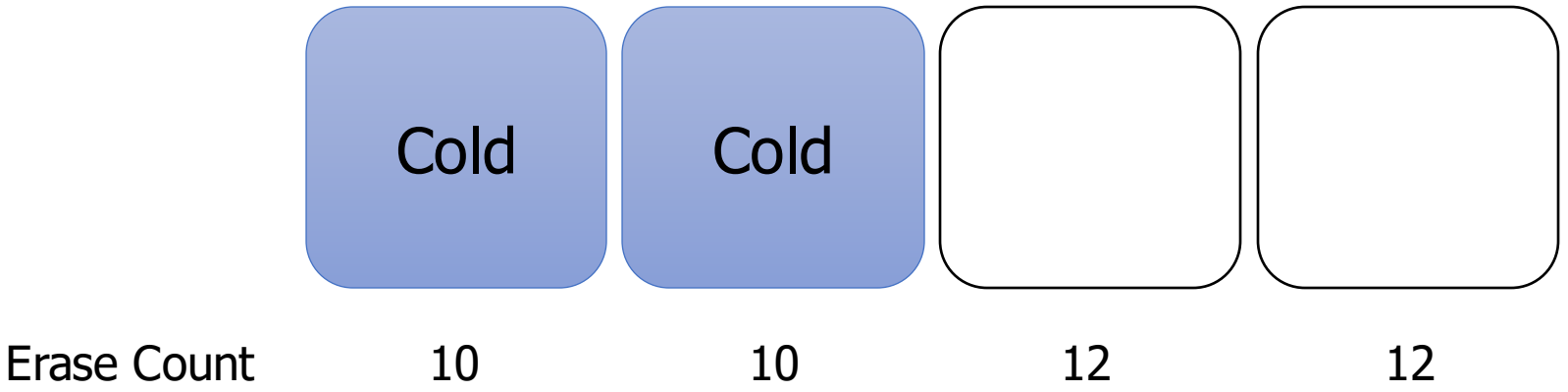
5.1. The Dilemma of FTL Wear-leveling

- Greedy Approach
  - FTL Cleaning Efficiency ↑
  - Uneven Wear of SSD Blocks



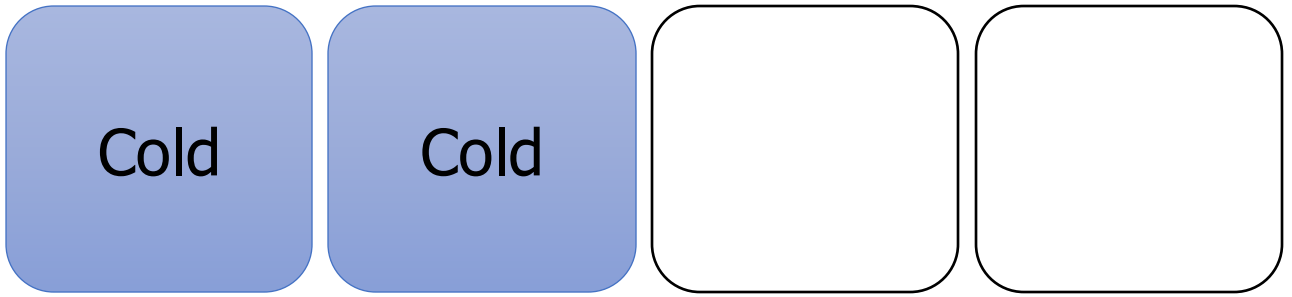
5.1. The Dilemma of FTL Wear-leveling

- Greedy Approach
  - FTL Cleaning Efficiency ↑
  - Uneven Wear of SSD Blocks



5.1. The Dilemma of FTL Wear-leveling

- Greedy Approach
  - FTL Cleaning Efficiency ↑
  - Uneven Wear of SSD Blocks



→ Repeats → 50% of Blocks Wear Fastly

5.2. The Solutions of FTL Wear-leveling

- Solution 1: Greedy but Rate-limiting
  - Block's Remaining Lifetime Drops (80% → 0%)
    → Probability of Recycling Also Drops (100% → 0%)
  - Problem: Simply Postpones

5.2. The Solutions of FTL Wear-leveling

- Solution 2: Greedy but Cold Data Migration
  - Cold Data → Old Blocks
  - Old Block = If Remaining Lifetime < retirementAge (ex. 85%)
  - Cold Data = ?

5.2. The Solutions of FTL Wear-leveling

- Solution 2: Greedy but Cold Data Migration
  - Cold Data → Old Blocks
  - Old Block = If Remaining Lifetime < retirementAge (ex. 85%)
  - Cold Data = ?

    → Tracking Data Update Frequency

    - Records Metadata → Migrates Together

5.2. The Solutions of FTL Wear-leveling

- Experiment
  - Greedy vs. Greedy + Rate-limiting + Migration
  - Environment: IOzone, 50 cycles

5.2. The Solutions of FTL Wear-leveling

- Experiment
  - Greedy vs. Greedy + Rate-limiting + Migration
  - Environment: IOzone, 50 cycles
- Result

	Mean Lifetime	Std.Dev.	Expired blocks
Greedy	43.82	13.47	223
+ Rate-limiting	43.82	13.42	153
+ Migration	43.34	5.71	0

Table 7: Block wear in IOzone

  - Wear-leveling is Solved

5.2. The Solutions of FTL Wear-leveling

- Experiment
  - Greedy vs. Greedy + Rate-limiting + Migration
  - Environment: IOzone, 50 cycles
- Result

	Mean Lifetime	Std.Dev.	Expired blocks
Greedy	43.82	13.47	223
+ Rate-limiting	43.82	13.42	153
+ Migration	43.34	5.71	0

Table 7: Block wear in IOzone

but
+ 4.7% Overhead
in I/O Latency

  - Wear-leveling is Solved

5.3. The Limit of Black-box Optimization

- Limitation
  - Which Data is Cold Data? OS Knows, but FTL Doesn't
    → History-based Approximations

5.4. Paradigm Shift: Opening the Box

- Suggestion: Sharing OS Information to FTL
  - Information 1: 'Which Blocks are Free'

5.4. Paradigm Shift: Opening the Box

- Suggestion: Sharing OS Information to FTL
  - Information 1: 'Which Blocks are Free'
    - Before OS Information
      - Stale Data: OS - Erased, FTL - Valid
        → Still Migrates When Cleaning → Write Amplification
      - FTL Knows 'Invalid' When Overwrite Occurs in Same LBA

5.4. Paradigm Shift: Opening the Box

- Suggestion: Sharing OS Information to FTL
  - Information 1: 'Which Blocks are Free'
    - After OS Information
      - GC Select Blocks With Less Valid Data
        → Valid Data Migration ↓
        → Cleaning Efficiency ↑

5.4. Paradigm Shift: Opening the Box

- Suggestion: Sharing OS Information to FTL
  - Information 2: 'Content Volatility of Data'
    - History-based Approximation of FTL

      → Approximation Overhead Disappears

6. Conclusion

6.1. Core Message

- The Importance of Interplay
  - Hardware: What is Theoretical I/O Throughput Limits?

6.1. Core Message

- The Importance of Interplay
  - Hardware: What is Theoretical I/O Throughput Limits?
  - Software: How to Tune the Mechanism of FTL?

6.1. Core Message

- The Importance of Interplay
  - Hardware: What is Theoretical I/O Throughput Limits?
  - Software: How to Tune the Mechanism of FTL?
  - Workload: What Characteristics Does This Workload Have?

6.1. Core Message

- The Importance of Interplay
  - Hardware: What is Theoretical I/O Throughput Limits?
  - Software: How to Tune the Mechanism of FTL?
  - Workload: What Characteristics Does This Workload Have?

- Sharing Information in OS to the FTL
  - Which Blocks are Free?
  - Content Volatility of Data

6.2. Significance

- Predicted Paradigm Shift in Enterprise Storage

Ref: 위키백과, 하드 디스크 드라이브, https://ko.wikipedia.org/wiki/%ED%95%98%EB%93%9C_%EB%94%94%EC%8A%A4%ED%81%AC_%EB%93%9C%EB%9D%BC%EC%9D%B4%EB%B8%8C
Ref: 삼성전자, https://semiconductor.samsung.com/kr/news-events/news/samsungs-980-nvme-ssd-combines-speed-and-affordability-to-set-a-new-standard-in-consumer-ssd-performance/

6.2. Significance

- Proved Universal Profits in SSD Design
  - Plane Interleaving
  - Moderate Overprovisioning

6.2. Significance

- Proved Universal Profits in SSD Design
  - Plane Interleaving
  - Moderate Overprovisioning
- Methodological Improvement
  - Trace-driven Simulator Instead of Hundreds of Hardware

6.2. Significance

- Proved Universal Profits in SSD Design
  - Plane Interleaving
  - Moderate Overprovisioning
- Methodological Improvement
  - Trace-driven Simulator Instead of Hundreds of Hardware
- Sementic Gap Solution in SSD
  - 'Which Blocks are Free' Information → Predicted TRIM

6.3. Limitations & Future Works

- Unfinished Model in Simulator
  - Details of Shared-control Ganging
  - More Refined Wear-leveling Data to Simulator

Thank you & Any Questions?